

Documentation: Simple Neural Processing Network with Random Genome

1 Overview

This document describes a simple feedforward neural processing network. It consists of:

- $N_S = 8$ sensory neurons
- $N_I = 4$ internal neurons
- $N_A = 5$ action neurons
- $N_C = 10$ encoded genome connections

Each connection (gene) is randomly generated and determines a weighted link from a source neuron to a sink neuron. The genome guides how information flows from the sensory neurons, through internal neurons, to the action neurons.

2 Softmax Function

The softmax function converts a vector $\mathbf{x} \in \mathbb{R}^n$ into a probability distribution:

$$\text{softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

3 Neuron State Vectors

Let:

- $\mathbf{s} \in \mathbb{R}^{N_S}$: Sensory neuron activations (random in $[-1, 1]$)
- $\mathbf{h} \in \mathbb{R}^{N_I}$: Internal neuron activations (initialized to 0)
- $\mathbf{a} \in \mathbb{R}^{N_A}$: Action neuron activations (initialized to 0)

4 Gene Encoding

Each gene is a 32-bit binary string, encoded as an 8-digit hexadecimal string. The structure is:

- Bit 0: Source type (0 = sensory, 1 = internal)
- Bits 1–7: Source neuron ID
- Bit 8: Sink type (0 = internal, 1 = action)
- Bits 9–15: Sink neuron ID
- Bits 16–31: Weight (interpreted as signed integer, scaled to $[-4.0, 4.0]$)

Weight Calculation

Let W be the 16-bit signed integer represented by bits 16–31. The weight is computed as:

$$w = \frac{W - 32768}{8192}$$

5 Signal Propagation

5.1 Phase 1: Sensory $\rightarrow$ Internal or Action

For each gene where the source is a sensory neuron (s_i):

$$\begin{aligned} \text{if sink is internal neuron } h_j : \quad & h_j += w \cdot s_i \\ \text{if sink is action neuron } a_k : \quad & a_k += w \cdot s_i \end{aligned}$$

5.2 Phase 2: Internal $\rightarrow$ Internal (Stacked)

For each gene from internal neuron h_i to internal neuron h_j :

$$h_j += w \cdot h_i$$

After all internal-to-internal updates, apply nonlinearity:

$$\mathbf{h} \leftarrow \tanh(\mathbf{h})$$

5.3 Phase 3: Internal $\rightarrow$ Action (Stacked)

For each gene from internal neuron h_i to action neuron a_k :

$$a_k += w \cdot h_i$$

Apply nonlinearity:

$$\mathbf{a} \leftarrow \tanh(\mathbf{a})$$

6 Action Selection

Compute softmax over the action neurons:

$$\mathbf{p} = \text{softmax}(\mathbf{a})$$

Sample action index i from the probability distribution $\mathbf{p}$:

$$\text{action} \sim \text{Categorical}(\mathbf{p})$$

7 Implementation Snippet

```
def softmax(x):
    f_x = np.exp(x) / np.sum(np.exp(x))
    return f_x

def generate_random_gene():
    return "".join(random.choices("0123456789abcdef", k=8))
```

```
sensory_neurons = np.random.uniform(-1, 1, N_SENSORY_NEURONS)
genome = [generate_random_gene() for _ in range(N_CONNECTIONS)]
...
action = np.random.choice(range(len(action_neurons)), p=softmax(action_neurons))
```

8 Notes

- Internal neuron processing is staged: sensory inputs are applied first, followed by internal-to-internal updates, then internal-to-action.
- The gene architecture allows for a compact, evolvable representation suitable for artificial life or agent control tasks.